

AI-Powered Image Editing System Overview

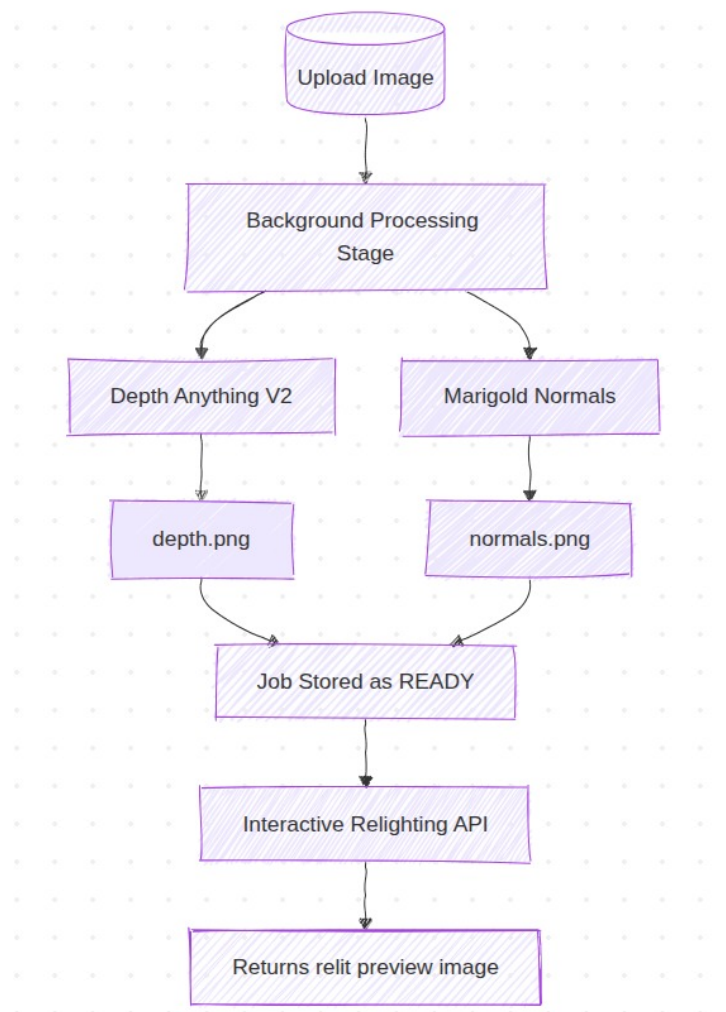
Team 42 (Mid Prep 2-Adobe)

1 Relighting feature documentation

This system enables AI-powered image relighting using depth maps and surface normals derived from a single 2D image. After the image is uploaded, the backend preprocesses it using two models:

- **Depth Anything V2** to extract depth information.
- **Marigold Normals Model** (diffusion-based) to estimate surface normals.

Once preprocessing is done, users can interactively relight the image by modifying lighting parameters such as light position, intensity, color, and shading properties — without re-running heavy inference. This architecture enables fast iterative relighting while ensuring high-quality scene geometry reconstruction.



1.1 Features

Feature	Description
Single inference per upload	Depth and surface normals are generated once to reduce processing cost.
Fast interactive relighting	Adjust light position, color, intensity, and material response in real time.
API-driven architecture	Designed for web/mobile UI integration.
Background worker support	Long computations run asynchronously to avoid blocking.

1.2 Preprocessing Components

Component	Model Used	Output File	Notes
Depth Estimation	Depth Anything V2 (ViT-L)	depth.png	Produces normalized 8-bit grayscale depth map
Normal Estimation	Marigold Normals v1-1 (Diffusion)	normals.png	High-quality surface normals from 2D image

1.3 API Endpoints

- **POST /upload** Uploads an image and starts preprocessing.
 - **Request:** multipart/form-data
 - * file: <image>
 - **Response:**

```
{
  "job_id": "f39a2e51d8124c89a8aa1f2b6936b889",
  "status": "queued"
}
```
- **GET /status/{job_id}** Poll this endpoint to check job progress. Possible statuses: queued, processing, ready, error.
- **POST /relight** Generates a relit image using previously computed depth & normals.

Field	Type	Default	Description
job_id	string	required	ID from upload
x, y	float	required	Light position (image-space)
z	float	required	Depth offset of light
color	hex string	#ffffff	Light color
intensity	float	2.2	Direct light power

Response: Returns a JPEG binary stream.

- **GET /download/{job_id}/{filename}** Download artifacts (depth.png, normals.png, orig.png).
- **DELETE /cleanup/{job_id}** Deletes all stored files and job metadata.

1.4 Relighting Model

The relighting operation uses a simplified physically-inspired shading model:

- Lambertian diffuse
- Blinn-Phong specular
- Distance-based falloff
- Shadow approximation using depth occlusion

Final light calculation:

$$\text{Final Light} = \text{Ambient} + (\text{Diffuse} + \text{Specular} \times \text{weight}) \times \text{falloff} \times (1 - \text{shadow})$$

$$\text{Output} = \text{Original pixels} \times \text{Final Light} \times \text{Light Color}$$

This enables realistic illumination transfer without requiring full 3D reconstruction.

1.5 Performance

Operation	Model Dependency	CPU Latency*
Depth estimation	Depth Anything V2	~2–5s
Normal estimation	Marigold (Diffusion)	~5–10s
Relighting	Math-only	<100ms

1.6 Advantages of Design

- Decouples expensive model inference from real-time rendering.
- Enables responsive UI control for lighting experiments.
- Supports future deployment on mobile or browser using ONNX or GPU inference.

2 Segmentation feature documentation

2.1 System Overview

The backend is implemented as a FastAPI service responsible for segmentation, cropping, and inpainting workflows. The system combines MobileSAM for point-based segmentation and LaMa for high-quality inpainting. Each request is tracked as an asynchronous job, allowing frontend clients to poll for status while processing continues in the background.

2.2 Model Pipeline

The processing pipeline consists of the following stages, executed sequentially:

1. **Image Upload** The client uploads an image, which is stored under a unique job identifier for tracking.
2. **Point-Based Segmentation** MobileSAM is invoked using a single (x, y) coordinate to produce a boolean mask representing the selected object.

3. **Cropping** A bounding box is extracted from the mask and used to generate an RGBA crop of the segmented object.
4. **Inpainting** LaMa receives the original image and enhanced mask to reconstruct the removed region. Full-image inpainting is performed after processing all selected objects.

2.3 Job Management

Each job maintains the following fields:

- `status`: queued, processing, ready, or error.
- `progress`: numeric estimate of pipeline completion level.
- `job_dir`: filesystem path containing artifacts.
- `successful_segmentations`: count of valid masks.

Jobs persist in memory and their artifacts reside inside a dedicated directory under `./jobs/`.

2.4 Key Utility Functions

- `segment_object(img_np, coordinates)` Runs MobileSAM inference to generate the segmentation mask.
- `crop_with_transparency(img_np, mask)` Produces an RGBA crop using the mask's bounding region.
- `inpaint_object(img_np, mask)` Performs LaMa inpainting with mask enhancement (dilation + Gaussian blur).
- `process_segmentation_job(...)` Coordinates all stages, updates job progress, and saves all generated artifacts.

2.5 API Endpoints

The service exposes the following endpoints:

- **POST /upload** Uploads an image and initializes a new job. Returns a `job_id` for future calls.
- **POST /segment** Starts segmentation and inpainting using a provided point coordinate. Accepts JSON: `{job_id, x, y}`.
- **GET /status/{job_id}** Returns real-time job status and progress.
- **GET /download/{job_id}/{filename}** Downloads generated artifacts such as `mask_i.png`, `cropped_i.png`, `inpainted_i.png`, or `full_inpainted.png`.
- **DELETE /cleanup/{job_id}** Deletes stored files and clears job metadata.
- **GET /health** Reports system health and model load status.

2.6 Model Loading

On startup, the backend loads:

- MobileSAM checkpoint (`vit_t`) for point-based segmentation.
- LaMa inpainting model for artifact-free reconstruction.

Both models run on GPU when available, otherwise CPU fallback is used.

2.7 Error Handling

The system records stack traces, sets job status to `error`, and preserves error details in the job dictionary. Missing files, invalid job IDs, and segmentation failures are gracefully reported through standardized HTTP exceptions.